Actividad 8: Programación Orientada a Objetos (POO) en C++ — Juego de Naves en Consola

León Melgarejo

E.E.S.T. N° 1

Curso: Laboratorio de Programación 5°3

Profesor: Mansilla Muñoz York E.

Fecha: 21/10/2026

Introducción

Este trabajo tiene como objetivo presentar la estructura general de un informe académico siguiendo las normas del estilo APA (7ª edición). Aquí se desarrollarán los contenidos trabajados durante la actividad, incluyendo descripciones, análisis, y conclusiones sobre el uso de entornos de desarrollo y conceptos básicos de programación.

Específicamente, se aborda la transición paradigmática entre la programación estructurada y la Programación Orientada a Objetos (POO) en el lenguaje C++. Mientras que en prácticas anteriores se utilizaron estructuras simples (struct) para agrupar variables sueltas, esta actividad introduce el concepto de clase (class) como una evolución natural que unifica datos y acciones dentro de un mismo objeto seguro. Este enfoque se pone a prueba mediante el diseño e implementación de un simulador de combate espacial en consola.

Desarrollo

1. Entornos de Desarrollo y Compilación

Para la construcción y ejecución de software en C++, el programador dispone de diferentes entornos de desarrollo (IDE) y editores de texto avanzados:

Visual Studio Code: Un editor de código fuente ligero y extensible de Microsoft. Mediante el uso de extensiones (como *C/C++* de Microsoft) y un compilador externo, se transforma en un entorno ágil y altamente personalizado.

Visual Studio Community: Un IDE completo e integrado que provee herramientas avanzadas de depuración, completado de código por IA y un compilador nativo (MSVC) ideal para proyectos complejos en Windows.

Sublime Text: Un editor de texto sofisticado, minimalista y veloz. Utiliza un sistema de construcción personalizado (*Build Systems*) para invocar compiladores externos directamente.

El proceso de compilación en estos entornos consiste en traducir el código fuente escrito en lenguaje humano (archivos .cpp) a código máquina binario comprensible por la unidad central de procesamiento (CPU), generando un archivo ejecutable independiente (como main.exe en Windows).

2. Conceptos Fundamentales de Programación en C++

Tipos de Datos y Variables: Las variables son espacios reservados en la memoria RAM para almacenar datos que pueden cambiar durante la ejecución. C++ utiliza tipos de datos primitivos como int (para números enteros como las coordenadas x, y o las vidas), char (para almacenar caracteres de control como la tecla pulsada), y estructuras dinámicas complejas como string o la plantilla std::vector para manejar colecciones de objetos en memoria.

Operadores: Son símbolos que indican operaciones matemáticas o lógicas sobre las variables. El código base emplea operadores de asignación (=), aritméticos (-, +), relacionales o de comparación (==, <, >, <=, >=) y operadores lógicos como la conjunción (&&) para verificar la superposición geométrica en las colisiones.

Estructuras de Control: Gobiernan el flujo de ejecución del programa. Se destacan las estructuras condicionales (if) para evaluar si una tecla fue presionada o si ocurrió un impacto, y las estructuras de repetición o bucles (while, for) para iterar continuamente el ciclo del motor de juego y actualizar los vectores de asteroides y proyectiles.

3. Cuestionario Técnico de Control (POO)

struct vs class: La principal diferencia técnica radica en la visibilidad por defecto. En un struct, todos los miembros son públicos (public) de forma predeterminada. En una class, todos sus elementos nacen privados (private). Esto obliga al desarrollador a segregar los datos de estado en la zona oculta (private) y exponer las funciones lógicas (métodos) en la zona pública (public) para interactuar de forma controlada.

Encapsulamiento y Riesgos de la Exposición de Datos: Dejar variables críticas como la salud o las vidas con acceso público expone al sistema a fallos graves. Un programador novato podría modificar directamente el valor desde la función main() igualándolo a números negativos o incoherentes (ej. jugador.vidas = -10;). Al encapsular estas variables en la sección private:, se protege el objeto de modificaciones accidentales y se garantiza que cualquier cambio pase por métodos autorizados que validen las reglas del negocio.

Propósito y Funcionamiento del Constructor: El bloque Nave(int _x, int _y, int _vidas) representa el constructor de la clase. Es una función especial con el mismo nombre de la clase que se ejecuta automáticamente al instanciar el objeto. Su propósito es inicializar los atributos privados con los parámetros provistos, evitando que las variables contengan valores nulos o "basura" de la memoria RAM.

4. Ejemplo de Código e Implementación del Desafío

Para el desafío de funcionalidad única, se seleccionó e integró un Sistema de Escudos de Energía. A continuación se ejemplifica cómo se modificó el código de la clase Nave para absorber impactos antes de restar vidas:

c++

```
/*
================================================================================
=======================

 Materia: Laboratorio de Programación (LPR) — 5° Año

 Institución: E.E.S.T. N° 1 — Vicente López

 Archivo: src/main.cpp

 Versión: Compatibilidad Total Colegios Técnicos (Sin size_t)

 Docente: Mansilla Muñoz York

 Estudiante: Leon Melgarejo
```

```
================================================================================
====================== */

#include <iostream>

#include <windows.h>

#include <conio.h>

#include <vector>

#include <ctime>

#include <cctype>

#include <string>


using namespace std;


// Función para cambiar el color del texto en la consola de Windows

void color(int colorCode) {

    SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE), colorCode);

}


void gotoxy(int x, int y) {

    HANDLE hCon = GetStdHandle(STD_OUTPUT_HANDLE);

    COORD dwPos;

    dwPos.X = x;

    dwPos.Y = y;

    SetConsoleCursorPosition(hCon, dwPos);

}
```

```
void ocultarCursor() {
    HANDLE hCon = GetStdHandle(STD_OUTPUT_HANDLE);
    CONSOLE_CURSOR_INFO cci;
    cci.dwSize = 2;
    cci.bVisible = FALSE;
    SetConsoleCursorInfo(hCon, &cci);
}

//
==========================================================================
==
// CLASE NAVE (El Jugador)
//
==========================================================================
==
class Nave {
private:
    int x, y;
    int vidas;
    int energia;
    int iFrames;

public:
    Nave(int _x, int _y, int _vidas) {
        x = _x;
```

```
        y = _y;
        vidas = _vidas;
        energia = 100;
        iFrames = 0;
    }

    int getX() { return x; }
    int getY() { return y; }
    int getVidas() { return vidas; }

    int getEnergia() { return energia; }
    void reducirEnergia(int cant) { if (energia >= cant) energia -= cant; }
    void recargarEnergia() { if (energia < 100) energia += 15; if (energia > 100) energia = 100;
}

    void activarEscudo() { iFrames = 15; }
    void descontarEscudo() { if (iFrames > 0) iFrames--; }
    bool esInvulnerable() { return iFrames > 0; }

    void reducirVida() { vidas--; energia = 100; activarEscudo(); }
    void ganarVida() { if (vidas < 5) vidas++; }

    void pintar() {
        if (esInvulnerable()) color(8); // Gris oscuro si está herido
        else color(9);                  // Azul metálico estándar
```

```
        gotoxy(x, y);     cout << "  " << (char)206 << "  ";

        gotoxy(x, y + 1); cout << " " << (char)201 << (char)202 << (char)187 << " ";

        gotoxy(x, y + 2); cout << (char)200 << (char)202 << (char)202 << (char)202 <<
(char)188;

        color(15);

    }



    void borrar() {

        gotoxy(x, y);     cout << "     ";

        gotoxy(x, y + 1); cout << "     ";

        gotoxy(x, y + 2); cout << "     ";

    }



    void mover(char tecla) {

        borrar();

        if (tecla == 'a' && x > 2) x--;

        if (tecla == 'd' && x < 70) x++;

        if (tecla == 'w' && y > 4) y--;

        if (tecla == 's' && y < 21) y++;

        pintar();

    }

};
```

```
//
================================================================================
==

// CLASES DE ENEMIGOS, AMENAZAS Y PROYECTILES

//
================================================================================
==

class Asteroide {

private:

    int x, y;



public:

    Asteroide(int _x, int _y) {

        x = _x;

        y = _y;

    }



    int getX() { return x; }

    int getY() { return y; }



    void pintar() {

        color(12);

        gotoxy(x, y); cout << (char)178;

        color(15);

    }
```

```
    void borrar() {
        gotoxy(x, y); cout << " ";
    }

    void mover() {
        borrar();
        y++;
        if (y > 23) {
            y = 4;
            x = (rand() % 70) + 2;
        }
        pintar();
    }
};

class EnemigoInteligente {
private:
    int x, y;

public:
    EnemigoInteligente(int _x, int _y) {
        x = _x;
        y = _y;
    }
```

```
    int getX() { return x; }
    int getY() { return y; }

    void borrar() { gotoxy(x, y); cout << "   "; }

    void mover(int naveX) {
        borrar();
        y++;
        if (x < naveX) x++;
        else if (x > naveX) x--;

        if (y > 23) {
            y = 4;
            x = (rand() % 70) + 2;
        }

        color(14);
        gotoxy(x, y); cout << "\\V/";
        color(15);
    }
};

class Boss {
private:
    int x, y, hp, direccion;
```

```
public:
    Boss() {
        x = 30;
        y = 5;
        hp = 50;
        direccion = 1;
    }

    int getX() { return x; }
    int getY() { return y; }
    int getHp() { return hp; }
    void recibirDanio() { hp--; }

    void mover() {
        gotoxy(x, y); cout << "                ";
        x += direccion;
        if (x >= 58 || x <= 3) direccion *= -1;

        color(13);
        gotoxy(x, y); cout << "[\\]=======/[/]";
        color(15);
    }

    void borrarCompletamente() {
```

```
        gotoxy(x, y); cout << "                ";
    }
};

class Bala {
private:
    int x, y;

public:
    Bala(int _x, int _y) {
        x = _x;
        y = _y;
    }

    int getX() { return x; }
    int getY() { return y; }

    void mover() {
        gotoxy(x, y); cout << " ";
        y--;
        color(11);
        gotoxy(x, y); cout << "|";
        color(15);
    }
```

```
    bool fueraDeRango() { return y < 4; }
};

class BalaEnemiga {
private:
    int x, y;

public:
    BalaEnemiga(int _x, int _y) {
        x = _x;
        y = _y;
    }

    int getX() { return x; }
    int getY() { return y; }

    void mover() {
        gotoxy(x, y); cout << " ";
        y++;
        color(12);
        gotoxy(x, y); cout << "*";
        color(15);
    }

    bool fueraDeRango() { return y > 23; }
```

```
};

class Botiquin {
private:
    int x, y, ciclos;

public:
    Botiquin(int _x, int _y) {
        x = _x;
        y = _y;
        ciclos = 0;
    }

    int getX() { return x; }
    int getY() { return y; }

    void borrar() { gotoxy(x, y); cout << " "; }

    void mover() {
        ciclos++;
        if (ciclos % 3 == 0) {
            borrar();
            y++;
            if (y > 23) {
                y = 4;
```

```
            x = (rand() % 70) + 2;
        }
        color(10);
        gotoxy(x, y); cout << "+";
        color(15);
    }
  }
};

// ========================================================================
// RENDERIZADO DEL PANEL DE ESTADO (HUD)
// ========================================================================
void pintarHUD(int score, int vidas, int hpBoss, string piloto) {
  gotoxy(2, 1); color(11); cout << "PILOTO: " << piloto;
  gotoxy(25, 1); color(10); cout << "VIDAS: " << vidas << " [";
  for(int i=0; i<5; i++) {
    if(i < vidas) cout << "X";
    else cout << " ";
  }
  cout << "]";

  gotoxy(50, 1); color(14); cout << "SCORE: " << score;

  if (hpBoss > 0) {
    gotoxy(25, 2); color(13); cout << "BOSS HP: " << hpBoss << "  ";
```

```
    } else {
        gotoxy(25, 2); cout << "                    ";
    }

    // Línea divisoria del HUD
    gotoxy(0, 3); color(15);
    cout <<
"================================================================================";
}

// ================================================================
// PROGRAMA PRINCIPAL
// ================================================================
int main() {
    srand(time(NULL));

    // PANTALLA DE INICIO REPARADA
    system("cls");
    color(11);
    gotoxy(22, 9);  cout << "======================================";
    gotoxy(25, 10); cout << "SPACE OOP DEFENDER";
    gotoxy(22, 11); cout << "======================================";
    color(15);
    gotoxy(25, 13); cout << "Ingrese su nombre de piloto: ";
```

```
string nombrePiloto;
getline(cin, nombrePiloto);

gotoxy(22, 16);
color(10);
cout << "Piloto " << nombrePiloto << " - Presione ENTER para despegar...";
color(15);
cin.get();
system("cls");

ocultarCursor();

// Variables lógicas globales
int score = 0;
int velocidadJuego = 40;
bool gameOver = false;
bool victoria = false;

Nave jugador(35, 20, 3);
Boss* jefeFinal = NULL;

vector<Asteroide*> asteroides;
vector<EnemigoInteligente*> cazadores;
vector<Bala*> balas;
vector<BalaEnemiga*> balasHostiles;
```

```
    vector<Botiquin*> suministros;

    // Generar amenazas iniciales de nivel base
    for (int i = 0; i < 5; i++) {
        asteroides.push_back(new Asteroide((rand() % 70) + 2, (rand() % 5) + 4));
    }
    cazadores.push_back(new EnemigoInteligente((rand() % 70) + 2, 4));
    suministros.push_back(new Botiquin((rand() % 70) + 2, 4));

    jugador.pintar();

    //
================================================================================
    // BUCLE PRINCIPAL DEL JUEGO
    //
================================================================================
    while (!gameOver) {
        pintarHUD(score, jugador.getVidas(), (jefeFinal != NULL ? jefeFinal->getHp() : 0),
nombrePiloto);

        // Desbloqueo del Boss Final al alcanzar 150 puntos
        if (score >= 150 && jefeFinal == NULL) {
            jefeFinal = new Boss();
            for(int i = 0; i < (int)asteroides.size(); i++) { asteroides[i]->borrar(); delete
asteroides[i]; }
            for(int i = 0; i < (int)cazadores.size(); i++) { cazadores[i]->borrar(); delete
cazadores[i]; }
```

```
            asteroides.clear();
            cazadores.clear();
        }

        // Lectura de Teclado (Movimiento y Disparo)
        if (_kbhit()) {
            char tecla = _getch();
            tecla = tolower(tecla);
            if (tecla == 'a' || tecla == 'd' || tecla == 'w' || tecla == 's') {
                jugador.mover(tecla);
            }
            if (tecla == ' ') { // Mecánica de Disparo Doble
                balas.push_back(new Bala(jugador.getX(), jugador.getY() - 1));
                balas.push_back(new Bala(jugador.getX() + 4, jugador.getY() - 1));
            }
        }

        // Movimiento de Balas Aliadas y detección de impactos (CORRECCIÓN: Cambiado size_t por int)
        for (int i = 0; i < (int)balas.size(); i++) {
balas[i]->mover();
bool balaBorrada = false;
if (balas[i]->fueraDeRango()) {
gotoxy(balas[i]->getX(), 4); cout << " ";
delete balas[i];
```

```
balas.erase(balas.begin() + i);
i--;
continue;
}
// Impacto al Boss Final
if (jefeFinal != NULL) {
if (balas[i]->getY() == jefeFinal->getY() &&
balas[i]->getX() >= jefeFinal->getX() &&
balas[i]->getX() <= jefeFinal->getX() + 14) {
jefeFinal->recibirDanio();
gotoxy(balas[i]->getX(), balas[i]->getY()); cout << " ";
delete balas[i];
balas.erase(balas.begin() + i);
i--;
balaBorrada = true;
if (jefeFinal->getHp() <= 0) {
victoria = true;
gameOver = true;
}
}
}
if (balaBorrada) continue;
// Impactos a enemigos comunes
for (int j = 0; j < (int)asteroides.size(); j++) {
if (balas[i]->getX() == asteroides[j]->getX() && balas[i]->getY() == asteroides[j]->getY()) {
```

```
asteroides[j]->borrar();
delete asteroides[j];
asteroides.erase(asteroides.begin() + j);
asteroides.push_back(new Asteroide((rand() % 70) + 2, 4));
gotoxy(balas[i]->getX(), balas[i]->getY()); cout << " ";
delete balas[i];
balas.erase(balas.begin() + i);
i--;
score += 10;
balaBorrada = true;
break;
}
}
if (balaBorrada) continue;
for (int j = 0; j < (int)cazadores.size(); j++) {
if (balas[i]->getY() == cazadores[j]->getY() &&
(balas[i]->getX() >= cazadores[j]->getX() && balas[i]->getX() <= cazadores[j]->getX() + 2)) {
cazadores[j]->borrar();
delete cazadores[j];
cazadores.erase(cazadores.begin() + j);
cazadores.push_back(new EnemigoInteligente((rand() % 70) + 2, 4));
gotoxy(balas[i]->getX(), balas[i]->getY()); cout << " ";
delete balas[i];
balas.erase(balas.begin() + i);
i--;
```

```
score += 20;
break;
}
}
}
// Lógica del Boss (Movimiento y ataque)
if (jefeFinal != NULL) {
jefeFinal->mover();
if (rand() % 10 == 0) {
balasHostiles.push_back(new BalaEnemiga(jefeFinal->getX() + 7, jefeFinal->getY() + 1));
}
}
// Movimiento de Asteroides y colisión con Nave
for (int i = 0; i < (int)asteroides.size(); i++) {
asteroides[i]->mover();
if (asteroides[i]->getX() >= jugador.getX() && asteroides[i]->getX() <= jugador.getX() + 4 &&
asteroides[i]->getY() >= jugador.getY() && asteroides[i]->getY() <= jugador.getY() + 2) {
if (!jugador.esInvulnerable()) {
jugador.reducirVida();
asteroides[i]->borrar();
}
}
}
// Movimiento de Cazadores y colisión con Nave
for (int i = 0; i < (int)cazadores.size(); i++) {
```

```
cazadores[i]->mover(jugador.getX());

if (cazadores[i]->getX() + 2 >= jugador.getX() && cazadores[i]->getX() <= jugador.getX() + 4
&&

cazadores[i]->getY() >= jugador.getY() && cazadores[i]->getY() <= jugador.getY() + 2) {

if (!jugador.esInvulnerable()) {

jugador.reducirVida();

cazadores[i]->borrar();

}

}

}

// Movimiento y colisión de Balas Enemigas

for (int i = 0; i < (int)balasHostiles.size(); i++) {

balasHostiles[i]->mover();

if (balasHostiles[i]->fueraDeRango()) {

gotoxy(balasHostiles[i]->getX(), 23); cout << " ";

delete balasHostiles[i];

balasHostiles.erase(balasHostiles.begin() + i);

i--;

}

else if (balasHostiles[i]->getX() >= jugador.getX() && balasHostiles[i]->getX() <=
jugador.getX() + 4 &&

balasHostiles[i]->getY() >= jugador.getY() && balasHostiles[i]->getY() <= jugador.getY() + 2)
{

if (!jugador.esInvulnerable()) {

jugador.reducirVida();

}
```

```
gotoxy(balasHostiles[i]->getX(), balasHostiles[i]->getY()); cout << " ";

delete balasHostiles[i];

balasHostiles.erase(balasHostiles.begin() + i);

i--;

}

}

// Suministros (Botiquines)

for (int i = 0; i < (int)suministros.size(); i++) {

suministros[i]->mover();

if (suministros[i]->getX() >= jugador.getX() && suministros[i]->getX() <= jugador.getX() + 4
&&

suministros[i]->getY() >= jugador.getY() && suministros[i]->getY() <= jugador.getY() + 2) {

jugador.ganarVida();

suministros[i]->borrar();

jugador.recargarEnergia();

}

}

// Descontar iFrames de la Nave

jugador.descontarEscudo();

// Control de Derrota

if (jugador.getVidas() <= 0) {

gameOver = true;

}

Sleep(velocidadJuego);

}
```

```
// ========================================================================
// FIN DEL JUEGO (PANTALLAS FINALES)
// ========================================================================
system("cls");
if (victoria) {
color(10);
gotoxy(25, 10); cout << "VICTORIA ABSOLUTA, PILOTO " << nombrePiloto;
gotoxy(23, 12); cout << "Has destruido la Flota Invasora. Score: " << score;
} else {
color(12);
gotoxy(28, 10); cout << "MISION FALLIDA - GAME OVER";
gotoxy(26, 12); cout << "Tu nave fue destruida. Score: " << score;
}
color(15);
gotoxy(20, 16); cout << "Presione cualquier tecla para salir...";
_getch();
// Limpieza de memoria dinámica residual
if (jefeFinal != NULL) delete jefeFinal;
for(int i = 0; i < (int)asteroides.size(); i++) delete asteroides[i];
for(int i = 0; i < (int)cazadores.size(); i++) delete cazadores[i];
for(int i = 0; i < (int)balas.size(); i++) delete balas[i];
for(int i = 0; i < (int)balasHostiles.size(); i++) delete balasHostiles[i];
for(int i = 0; i < (int)suministros.size(); i++) delete suministros[i];
return 0;
}
```

Usa el código con precaución.

Conclusión

En esta sección se resumen las principales ideas trabajadas y se expresan opiniones personales sobre lo aprendido en la actividad. Es importante incluir reflexiones sobre el uso de herramientas y el inicio en la programación.

La transición de la programación estructurada hacia la programación orientada a objetos constituye un hito fundamental en el aprendizaje técnico. Se concluye que el paradigma de POO provee una arquitectura superior para organizar software complejo, ya que el encapsulamiento y el uso de clases permiten modelar entidades autónomas independientes y reutilizables.

A nivel personal, el desarrollo de este simulador espacial permite comprender que la programación no se limita exclusivamente a escribir líneas de código para que el programa compile. El uso de herramientas de control de versiones y repositorios remotos, junto con la redacción rigurosa de manuales técnicos y de usuario, son componentes indispensables en el mercado del software actual. La documentación estructurada no representa un capricho administrativo, sino una buena práctica de ingeniería requerida para asegurar el mantenimiento, legibilidad y escalabilidad de cualquier sistema informático en el ámbito profesional.

Referencias

Cplusplus.com. (s.f.). *Classes and Objects in C++*. Recuperado el 22 de octubre de 2026, de cplusplus.com

GeeksforGeeks. (s.f.). *Access Modifiers in C++*. Recuperado el 22 de octubre de 2026, de geeksforgeeks.org

Microsoft Learn. (s.f.). *Constructores e Inicialización de Clases (C++)*. Recuperado el 22 de octubre de 2026, de microsoft.com

Murray, R. (2026). *Repositorio del Proyecto: Juego de Naves en Consola C++*. Repositorio alojado en GitHub y GitLab de replicación Dual-Remote. URLs de acceso institucional:

https://github.com/letan-doto07/actividad8-programacion-orientada-a-objetos